

MODÜL 1

Angular & Modern Frontend Temelleri

Angular v21 Eğitim Programı

Seviye:	Başlangıç · Sıfır → Junior
Ders sayısı:	5 ders
Tahmini süre:	1-2 hafta
Önkoşul:	TypeScript temel bilgisi

Hazırlayan: Claude
Versiyon: Angular v21 (Kasım 2025)

İçindekiler

Modül 1 — işlenen modül. Diğer modüller eğitim programı boyunca işlenecek.

Başlangıç · Sıfır → Junior

► Modül 1 — Angular & Modern Frontend Temelleri

→ Modül 1'e Giriş

→ Ders 1.1 — Frontend Frameworks: Neden Var?

- ↳ Web'in Tarihsel Hikayesi
- ↳ Vanilla JS'in Sınırları
- ↳ SPA (Single Page Application)
- ↳ Component-Based Architecture
- ↳ Reactivity Nedir?
- ↳ Vanilla JS vs Framework Karşılaştırması
- ↳ Bu Eğitimde Kullanılacak Felsefe
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.2 — Angular Tarihçesi & Felsefesi

- ↳ AngularJS — Her Şeyin Başlangıcı
- ↳ Büyük Yıkım ve Yeniden Doğuş — Angular 2
- ↳ Angular'ın Felsefesi — "Opinionated"
- ↳ Angular'ın Versiyon Tarihi
- ↳ Angular Kim Tarafından Kullanılıyor?
- ↳ Angular'ın Geleceği
- ↳ Bu Eğitimde Neden Angular v21?
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.3 — Angular vs React vs Vue (Kod Karşılaştırması)

- ↳ Karşılaştırma Senaryosu
- ↳ React Versiyonu
- ↳ Vue Versiyonu
- ↳ Angular Versiyonu (v21)
- ↳ Yan Yana Karşılaştırma
- ↳ "Bu Kod Bana Ne Söylüyor?"
- ↳ "Hangisini Öğrenmeliyim?"
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.4 — Angular v21'in "Big Picture" Yenilikleri

- ↳ v21 Bir "Geçiş Versiyonu" Değil
- ↳ Zoneless Change Detection
- ↳ Signal Forms
- ↳ Angular Aria
- ↳ Vitest — Karma'nın Sonu
- ↳ MCP Server — AI Çağına Entegrasyon
- ↳ Tailwind CSS

- ↳ Diğer Önemli Yenilikler
- ↳ Bu Yenilikler Senin İçin Ne Demek?
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Ders 1.5 — Geliştirme Ortamı Kurulumu

- ↳ Geliştirme Ortamımızda Neler Olacak?
- ↳ Node.js Kurulumu
- ↳ Angular CLI Kurulumu
- ↳ VS Code Kurulumu ve Uzantılar
- ↳ VS Code Ayarları
- ↳ Git Kurulumu
- ↳ Angular DevTools Kurulumu
- ↳ İlk Angular Projemizi Oluşturma
- ↳ Hızlı Bir Düzenleme — Hot Reload Görelim
- ↳ Angular DevTools'u İlk Kez Kullanma
- ↳ Geliştirme Workflow'u
- ↳ Sık Karşılaşılan Problemler
- ↳ Pratik Komutlar Sözlüğü
- ↳ Mini Quiz
- ↳ Anti-Patterns
- ↳ Bu Dersten Çıkarılacaklar

→ Modül 1 Özeti

- Modül 2 — TypeScript Refresher (Angular için)
 - 2.1 Temel tipler & interfaces
 - 2.2 Generics
 - 2.3 Decorators (kavramsal)
 - 2.4 Utility types
 - 2.5 Type guards & narrowing
 - 2.6 Mini proje: TypeScript bootcamp
- Modül 3 — İlk Angular Uygulaması, CLI & DevTools
 - 3.1 ng new ile proje oluşturma
 - 3.2 Proje yapısı anatomisi
 - 3.3 Angular CLI komutları
 - 3.4 app.config.ts & bootstrapping
 - 3.5 Angular DevTools
 - 3.6 MCP Server (v21 yeniliği)
 - 3.7 Mini proje: Counter
- Modül 4 — Component'lar & Modern Template Syntax
 - 4.1 Component anatomisi
 - 4.2 Standalone components
 - 4.3 Data binding 4 çeşidi
 - 4.4 Modern control flow (@if, @for, @switch)
 - 4.5 @let ile template değişkenleri
 - 4.6 input(), output(), model()
 - 4.7 View encapsulation
 - 4.8 Mini proje: Todo App

- Modül 5 — Signals & Reactivity
 - 5.1 Signal nedir? (teori)
 - 5.2 Signal API tam referans
 - 5.3 linkedSignal (v19+)
 - 5.4 resource API (v19+)
 - 5.5 Signal queries (viewChild, viewChildren)
 - 5.6 Mini proje: Reactive filter & search
- Modül 6 — Directives & Pipes
 - 6.1 Directive türleri
 - 6.2 Custom attribute directive
 - 6.3 Host directives
 - 6.4 Built-in pipes
 - 6.5 Custom pipe
 - 6.6 Mini proje: Highlight & format library
- Modül 7 — Services & Dependency Injection
 - 7.1 Service nedir?
 - 7.2 inject() vs constructor injection
 - 7.3 Provider scope (hierarchical injection)
 - 7.4 InjectionToken & multi-providers
 - 7.5 Modern provider patterns: provideX()
 - 7.6 Mini proje: Pluggable logger service
- Modül 8 — Project Structure & Feature-Based Architecture
 - 8.1 Neden klasör yapısı önemli?
 - 8.2 Feature-based architecture
 - 8.3 core / shared / features ayrımı
 - 8.4 Bağımlılık kuralları
 - 8.5 Index files (barrel exports)
 - 8.6 Path aliases (tsconfig.json)
 - 8.7 ESLint rules ile mimariyi korumak
 - 8.8 Mini proje: Project skeleton
- Modül 9 — Routing & Navigation
 - 9.1 Temel routing
 - 9.2 Route parameters & component input binding
 - 9.3 Functional guards
 - 9.4 Functional resolvers
 - 9.5 Lazy loading routes
 - 9.6 Nested routes
 - 9.7 Mini proje: Multi-role dashboard
- Modül 10 — RxJS Deep-Dive
 - 10.1 Observable: stream over time
 - 10.2 Operators: 4 kategori
 - 10.3 Higher-order mapping operators (kritik)
 - 10.4 Subjects (Subject, BehaviorSubject, ReplaySubject)
 - 10.5 Hot vs cold observables
 - 10.6 Error handling
 - 10.7 Memory leak prevention
 - 10.8 RxJS + signals interop
 - 10.9 Mini proje: Smart search

- Modül 11 — HTTP & Server Communication
 - 11.1 HttpClient setup
 - 11.2 CRUD işlemleri
 - 11.3 Functional interceptors
 - 11.4 httpResource() (v19+ modern yaklaşım)
 - 11.5 File upload & download
 - 11.6 HTTP caching patterns
 - 11.7 Mini proje: Blog API client
- Modül 12 — State Management (NgRx Signal Store)
 - 12.1 Hangi araç ne zaman?
 - 12.2 Service + signal ile basit state
 - 12.3 NgRx Signal Store setup
 - 12.4 Signal Store anatomisi
 - 12.5 Component'ta kullanım
 - 12.6 withEntities (entity management)
 - 12.7 Feature store (component-scoped)
 - 12.8 Mini proje: Shopping cart
- Modül 13 — Angular Material & CDK
 - 13.1 Neden UI library?
 - 13.2 UI library karşılaştırması
 - 13.3 Angular Material setup
 - 13.4 Theming (Material Design 3)
 - 13.5 Sık kullanılan component'lar
 - 13.6 MatDialog (modal)
 - 13.7 Angular CDK nedir?
 - 13.8 CDK drag & drop
 - 13.9 CDK overlay (tooltip, popover)
 - 13.10 CDK virtual scrolling
 - 13.11 CDK layout (responsive)
 - 13.12 Mini proje: Material admin dashboard

Olgunlaşma • Mid → Senior

- Modül 14 — Forms (Reactive + Signal Forms)
 - 14.1 Forms yaklaşımları
 - 14.2 Reactive Forms temelleri
 - 14.3 Typed forms (v14+)
 - 14.4 FormArray (dynamic lists)
 - 14.5 Custom validators
 - 14.6 Cross-field validation
 - 14.7 Conditional validation
 - 14.8 Signal Forms (v21 experimental)
 - 14.9 Mini proje: Multi-step wizard
- Modül 15 — Animations
 - 15.1 Setup
 - 15.2 Trigger, state, transition
 - 15.3 :enter & :leave
 - 15.4 Stagger (liste animasyonları)

- 15.5 Route animations
- 15.6 Reduced motion desteği
- 15.7 Mini proje: Animated card gallery
- Modül 16 — Lazy Loading & @defer
 - 16.1 3 seviyeli lazy loading
 - 16.2 @defer tetikleyicileri
 - 16.3 Tam @defer sözdizimi
 - 16.4 Prefetch stratejileri
 - 16.5 NgOptimizedImage
 - 16.6 Mini proje: E-commerce product page
- Modül 17 — WebSockets & Real-Time Communication
 - 17.1 Real-time: 3 yaklaşım
 - 17.2 RxJS WebSocket subject
 - 17.3 Reconnection strategy
 - 17.4 Server-sent events (SSE)
 - 17.5 Socket.IO entegrasyonu
 - 17.6 Mini proje: Real-time chat
- Modül 18 — i18n (Internationalization)
 - 18.1 Yaklaşımlar
 - 18.2 Transloco (önerilen)
 - 18.3 Locale-aware pipes
 - 18.4 RTL support
 - 18.5 Pluralization & ICU messages
 - 18.6 Mini proje: Multi-language blog
- Modül 19 — Accessibility (a11y) & Angular Aria
 - 19.1 Neden a11y?
 - 19.2 WCAG 2.2 (POUR)
 - 19.3 Semantic HTML first
 - 19.4 ARIA attributes
 - 19.5 Angular Aria (v21 yeniliği)
 - 19.6 Focus management
 - 19.7 Klavye navigasyonu
 - 19.8 Testing
 - 19.9 Mini proje: Accessible form & modal
- Modül 20 — Security
 - 20.1 Web security temelleri (OWASP)
 - 20.2 XSS koruması
 - 20.3 DomSanitizer
 - 20.4 Content security policy
 - 20.5 CSRF protection
 - 20.6 JWT storage stratejileri
 - 20.7 Dependency security
 - 20.8 Sensitive data handling
 - 20.9 Mini proje: Secure auth flow
- Modül 21 — Testing (Vitest + Playwright)
 - 21.1 Test pyramidi
 - 21.2 Vitest setup

- 21.3 Component testing
- 21.4 Service mocking
- 21.5 Signal testing
- 21.6 Signal Store testing
- 21.7 E2E with Playwright
- 21.8 Mini proje: %80+ coverage

Profesyonel • Senior

- Modül 22 — Performance & Change Detection
 - 22.1 Change detection: zone vs zoneless
 - 22.2 OnPush vs default
 - 22.3 Memory leaks
 - 22.4 Bundle size optimization
 - 22.5 Track-by optimization
 - 22.6 Performance profiling
 - 22.7 Web vitals
 - 22.8 Mini proje: Performance audit
- Modül 23 — Error Handling & Logging
 - 23.1 Neden önemli?
 - 23.2 Global ErrorHandler
 - 23.3 HTTP error interceptor
 - 23.4 User-friendly error UI
 - 23.5 Sentry entegrasyonu
 - 23.6 Logging service pattern
 - 23.7 Performance monitoring
 - 23.8 Mini proje: Error-resilient app
- Modül 24 — Environment Config & Build Configurations
 - 24.1 Environment files
 - 24.2 angular.json build configurations
 - 24.3 Runtime configuration
 - 24.4 Feature flags
 - 24.5 Build optimization
 - 24.6 Source maps stratejisi
 - 24.7 Mini proje: Multi-environment setup
- Modül 25 — SSR, Hydration & Web Vitals
 - 25.1 Neden SSR?
 - 25.2 Setup
 - 25.3 Hydration
 - 25.4 Component-level hydration (v21)
 - 25.5 Server-only / client-only code
 - 25.6 SEO meta tags
 - 25.7 Mini proje: SEO-optimized blog
- Modül 26 — PWA (Progressive Web Apps)
 - 26.1 PWA nedir?
 - 26.2 Setup
 - 26.3 Service worker caching
 - 26.4 Update notifications

- 26.5 Push notifications
- 26.6 Install prompt
- 26.7 Offline detection
- 26.8 Mini proje: Offline-first notes app

Expert · Senior → Expert

- Modül 27 — Microfrontend Architecture
 - 27.1 Microfrontend nedir?
 - 27.2 Module Federation (Webpack)
 - 27.3 Native Federation (modern alternatif)
 - 27.4 Inter-MFE communication
 - 27.5 @defer + Module Federation
 - 27.6 Mini proje: E-commerce microfrontend
- Modül 28 — Design Patterns
 - 28.1 Smart vs dumb components
 - 28.2 Repository pattern
 - 28.3 Facade pattern
 - 28.4 Strategy pattern
 - 28.5 Observer pattern (built-in)
 - 28.6 Mini proje: Architecture refactor
- Modül 29 — CI/CD & Deployment
 - 29.1 Build pipeline anatomisi
 - 29.2 GitHub Actions
 - 29.3 Docker
 - 29.4 Vercel / Netlify deployment
 - 29.5 Cloudflare Pages
 - 29.6 Self-hosting (Nginx + VPS)
 - 29.7 SSR deployment stratejileri
 - 29.8 Environment variables in CI
 - 29.9 Mini proje: Full CI/CD pipeline

Capstone · Final Proje

- Modül 30 — Capstone — Task Management Platform
 - H1 Mimari tasarım + project skeleton
 - H2 Shell + auth + routing + Material setup
 - H3 MFE-Projects (CRUD, formlar)
 - H4 MFE-Tasks (@defer, drag-drop, real-time)
 - H5 MFE-Reports (dashboard, virtual scroll)
 - H6 State management (NgRx Signal Store)
 - H7 i18n + a11y + security audit
 - H8 Testing + error handling + monitoring
 - H9 SSR + PWA
 - H10 CI/CD + production deployment

Modül 1'e Hoş Geldin

Bu modülde Angular dünyasının kapısını aralayacağız. Henüz çok fazla kod yazmayacağız — önceliğimiz kavramları doğru anlamak. Bir framework'ü öğrenmek, sadece syntax'ını ezberlemek değil; o framework'ün neden var olduğunu, hangi problemleri çözdüğünü ve nasıl düşündüğünü anlamaktır.

Bu modülün sonunda:

- ✓ Frontend framework'lerinin neden var olduğunu anlayacaksın
- ✓ Angular'ın felsefesini ve diğer framework'lerden farkını bileceksin
- ✓ Angular v21'in büyük yeniliklerini tanıyacaksın
- ✓ Geliştirme ortamını kurmuş ve ilk uygulamayı çalıştırmış olacaksın



Öğrenme Felsefesi

Bu eğitimde "Neden?" sorusunu sürekli soracağız. Bir konsept gördüğünde sadece "nasıl yazılır" değil, "neden böyle yazılır, alternatifleri ne, ne zaman kullanılır" sorularını da kavramaya çalışacağız.

MODÜL 1 — DERS 1.1

Frontend Frameworks: Neden Var?

*Bu derste Angular'a kod yazmadan önce asıl soruyu cevaplayacağız:
"Neden framework'lere ihtiyaç var? Vanilla JS ile yapamaz mıyız?"*



Hedef

Bu sorunun cevabı, sonraki tüm derslerin temelini oluşturuyor. Framework'lerin neden var olduğunu, hangi problemleri çözdüklerini kavrayacağız.

1

Web'in Tarihsel Hikayesi

Angular'ı anlamak için önce nereden geldiğimizi anlamalıyız.

❑ Taş Devri: Static HTML (1990'lar)

İlk web siteleri sadece HTML dosyalarıydı. Sunucu sana bir HTML gönderiyordu, sen okuyordun, başka bir sayfaya gitmek için yeni bir HTML talep ediyordun.

```
Kullanıcı: "Anasayfayı göster"  
Sunucu: <html>...</html> gönderir  
Kullanıcı bağlantıya tıklar  
Sunucu: Yeni <html>...</html> gönderir
```

Sorun: Her tıklamada tüm sayfa yeniden yükleniyor. Çok yavaş, çok kötü deneyim.

❑ Demir Devri: jQuery & DOM Manipulation (2000'ler)

JavaScript geldi. Artık sayfayı yeniden yüklemekten DOM'u (Document Object Model) değiştirebiliyoruz.

```
// jQuery dönemi  
$('#user-name').text('John');  
$('#user-list').append('<li>Yeni kullanıcı</li>');  
$('.delete-btn').click(function() {  
  $(this).parent().remove();  
});
```

Bu büyük bir devrimdi! Ama bir sorun var...

❑ Cehennem: Spaghetti Code (2010'lar)

Uygulamalar büyüdükçe DOM manipülasyonu kabusla dönüştü. Senaryo: Bir e-ticaret sitesi düşün. Kullanıcı sepete ürün ekliyor. Sen şunları manuel

güncellemek zorundasın:

```
function addToCart(product) {  
  // 1. Sepet sayısını güncelle (header'da)  
  $('#cart-count').text(parseInt($('#cart-count').text()) + 1);  
  
  // 2. Sepet listesine ekle (sidebar'da)  
  $('#cart-list').append('<li>${product.name}</li>');  
  
  // 3. Toplam fiyatı hesapla (footer'da)  
  let total = parseFloat($('#total').text()) + product.price;  
  $('#total').text(total.toFixed(2));  
  
  // 4-8. Notification, empty mesajı, button enable, localStorage, anal  
  ytics...  
}
```

Görüyor musun problemi? Tek bir state değişikliği için, DOM'un 8 farklı yerini manuel olarak senkronize ediyorsun. Hangi birini unutursan UI bozulur. Buna "imperative programming" deniyor: "Şunu yap, şunu yap, şunu yap..."

□ Aydınlanma: Declarative UI (2013+)

Birisi şu fikri attı: "Ya UI, state'in bir fonksiyonu olsa? Ben state'i değiştirsem, UI kendi kendini güncellese?"

```
UI = f(state)
```

İşte modern frontend framework'leri (React, Vue, Angular) bu fikrin ürünü.

2 Vanilla JS'in Sınırları (Somut Örneklerle)

Belki "Bence vanilla JS ile her şey yapılabilir" diye düşünüyor olabilirsin. Doğru, yapılabilir. Ama maliyeti ne?

Sorun 1: State Senkronizasyonu

Tek state değişikliği = onlarca DOM güncellemesi. Vanilla JS'in en büyük problemi.

Sorun 2: Component Reusability

Bir component'i 5 sayfada kullanırsan, kopyala-yapıştır yapmak zorundasın. Tasarım değişince hepsini tek tek güncelle.

Sorun 3: Event Handling Cehennemi

AJAX ile DOM'a yeni element eklendiğinde event listener'ları yeniden attach etmek gerekiyor. Karmaşık.

Sorun 4: Routing

Tek sayfalık uygulama yapmak için URL hash'leri, History API, component switching — hepsini kendin yazmalısın.

Sorun 5: Form Validation

10 alanlı bir form için 300+ satır kod. Framework ile 30 satır.

3

SPA (Single Page Application) Konsepti

MPA (Multi Page Application) — Geleneksel

Anasayfa.html → Tıkla → Profil.html → Tıkla → Ayarlar.html
(yenile) (yenile) (yenile)

Her tıklamada: tüm HTML, CSS, JS yeniden indirilir. Tüm sayfa render edilir. Yavaş.

SPA — Single Page Application

Anasayfa → URL değişir → Profil bileşenini yükle → Ayarlar bileşenini yükle
(sayfa yenilenmez!)

Tek bir HTML dosyası var (index.html). JavaScript, hangi bileşenin gösterileceğine karar veriyor. Sayfalar arası geçiş anlık. Mobil uygulama gibi hissettirir.



SPA'nın Avantajları

- Hızlı navigasyon
- Mobil app benzeri deneyim
- Background'da veri çekme (loading state)
- Smooth transitions



SPA'nın Dezavantajları

- İlk yükleme yavaş (tüm JS bundle'ı indirilir)
- SEO problemleri (bot'lar JS çalıştıramayabilir)
- İlk paint geç



Spoiler

Bu dezavantajları çözmek için SSR (Server-Side Rendering) var. Modül 25'te işleyeceğiz.

4 Component-Based Architecture

Modern framework'lerin temeli component'ler. Analoji: Lego tuğlası. Her tuğla kendi başına bir şey, ama birleştirence ev olur.

Component'in 3 Bileşeni

1. State (veri):

```
class UserCardComponent {  
  name = 'John Doe';  
  age = 30;  
  isOnline = true;  
}
```

2. Template (görünüm):

```
<div class="card">  
  <h3>{{ name }}</h3>  
  <p>Yaş: {{ age }}</p>  
  @if (isOnline) {  
    <span class="badge">Online</span>  
  }  
</div>
```

3. Style (stil):

```
.card {  
  padding: 1rem;  
  border-radius: 8px;  
}  
.badge {  
  background: green;  
  color: white;  
}
```

Component'lerin Süper Güçleri

Reusability: 5 farklı yerde kullan. Tasarım değişti? Tek dosyayı güncelle.

Encapsulation: Component'in CSS'i sadece o component'i etkiler.

Composition: Küçük bileşenler birleşerek karmaşık UI'lar oluşturur. Her parça bağımsız test edilebilir.

5 Reactivity Nedir?

Bu modern framework'lerin kalbi. Bu kavramı doğru anlamak çok kritik.

Klasik Programlama (Imperative)

```
let a = 5;
let b = 10;
let total = a + b; // total = 15

a = 20;
console.log(total); // 15! Çünkü total'i tekrar hesaplamadık
```

Reactive Programlama

```
let a = signal(5);
let b = signal(10);
let total = computed(() => a() + b()); // total = 15

a.set(20);
console.log(total()); // 30! Otomatik güncellendi.
```



Excel Analojisi

Reactivity'yi anlamak için Excel düşün:

A1 = 5, A2 = 10, A3 = =A1+A2 (formül)

A3 hücresi 15 olur. A1'i 20 yaparsan, A3 otomatik 30 olur. Çünkü A3, A1'e bağlı.

İşte reactivity bu. UI'nın state'e Excel hücrelerinin formüllerine bağlı olduğu gibi bağlı olması.

Angular'da Reactivity

```
@Component({
  template: `
    <input [(ngModel)]="name" />
    <p>Merhaba, {{ name() }}!</p>
    <p>Adın {{ name().length }} karakter.</p>
  `,
})
export class GreetingComponent {
  name = signal('Dünya');
}
```

Kullanıcı input'a yazdığı anda: signal güncellenir, UI otomatik güncellenir. Sen tek satır DOM kodu yazmadın!

6 Vanilla JS vs Framework — Karşılaştırma

Aynı küçük "Counter" uygulamasını iki şekilde yazalım:

Vanilla JS Versiyonu

```
<div id="app">
  <p>Sayı: <span id="count">0</span></p>
  <button id="increment">+1</button>
  <button id="decrement">-1</button>
</div>

<script>
  let count = 0;
  const countEl = document.getElementById('count');

  document.getElementById('increment').addEventListener('click', () =>
  {
    count++;
    countEl.textContent = count; // Manuel DOM update
  });

  document.getElementById('decrement').addEventListener('click', () =>
  {
    count--;
    countEl.textContent = count; // Yine manuel
  });
</script>
```

Sorunları: 3 farklı yerde DOM update kodu var. Yeni buton ekleyince yine manuel update gerekecek. "Sayı negatifse kırmızı" gibi kural eklenince DOM güncelleme dağınık.

Angular Versiyonu

```
@Component({
  selector: 'app-counter',
  template: `
    <p [class.negative]="count() < 0">Sayı: {{ count() }}</p>
    <button (click)="count.update(n => n + 1)">+1</button>
    <button (click)="count.update(n => n - 1)">-1</button>
    <button (click)="count.set(0)">Reset</button>
  `,
  styles: [`.negative { color: red; }`],
})
export class CounterComponent {
  count = signal(0);
}
```

Avantajları: State ve UI net şekilde ayrı. Manuel DOM update yok. "Negatifse kırmızı" kuralı declarative. Test edilmesi çok kolay.

7

Bu Eğitim Boyunca Kullanacağımız Felsefe

Bu eğitim boyunca 3 prensibi içselleştireceksin:

1. State'i Tek Yerde Tut

```
// ❌ Kötü – state DOM'da saklı
const count = parseInt(document.getElementById('count').textContent);

// ✅ İyi – state component'te, DOM sadece görünüm
count = signal(0);
```

2. UI = f(state)

UI, state'in bir fonksiyonu olmalı. State değişince UI otomatik güncellensin.

3. Component = Bağımsız Birim

Her component:

- Kendi state'ini yönetir
- Kendi template'ine sahiptir
- Kendi CSS'i kendisini etkiler
- Tek başına test edilebilir
- Tek başına anlamlıdır

📝 Mini Quiz

- **Vanilla JS ile yapılabilecek bir şey neden bir framework gerektirir?**

Cevap: Yapılabilir ile verimli, sürdürülebilir, hatasız yapılabilir farklı şeylerdir. Framework'ler state senkronizasyonu, component reusability, routing, forms gibi karmaşık işleri standardize eder.

- **SPA ile MPA arasındaki temel fark nedir?**

Cevap: MPA'da her sayfa geçişi yeni bir HTML talebi yapar (sayfa yenilenir). SPA'da tek bir HTML var; navigasyon JavaScript ile yönetilir, sayfa yenilenmez.

- **Imperative ve declarative programlama arasındaki fark nedir?**

Cevap: Imperative: "Şunu yap, şunu yap." (DOM'u manuel güncelle). Declarative: "Şu state şu UI'a karşılık gelir." (State'i değiştir, UI otomatik güncellenir).

- **Reactivity'yi Excel analojisiyle anlat.**

Cevap: Excel'de bir hücreye $=A1+A2$ formülü yazınca, A1 veya A2 değişince o hücre otomatik güncellenir. Reactivity de tam olarak bu.

- **Component'in 3 ana bileşeni nedir?**

Cevap: State (veri/logic — TypeScript), Template (görünüm — HTML), Style (stil — CSS/SCSS).

⚠️ Anti-Patterns

Daha kod yazmadık ama düşünme şekli olarak şunlardan kaçın:

- ✗ "Framework gereksiz, vanilla JS ile yapılır" — Yapılır, ama maliyeti devasa.
- ✗ DOM'u state olarak kullanmak — DOM görünüm. State component'te tutulur.
- ✗ "Manuel her şeyi kontrol ederim" — Reactive sistem 1000 kat daha güvenilir.
- ✗ jQuery alışkanlıklarını taşımak — `$('#elem').click(...)` mantığı modern Angular'da yok.

📝 Bu Dersten Çıkarman Gerekenler

- ✓ Frontend framework'lerinin neden var olduğunu biliyorum
- ✓ SPA'nın ne olduğunu ve geleneksel web'den farkını biliyorum
- ✓ Component-based mimarinin avantajlarını anlıyorum
- ✓ Reactivity konseptini Excel analojisiyle açıklayabiliyorum
- ✓ "UI = f(state)" felsefesini içselleştirdim

MODÜL 1 — DERS 1.2

Angular Tarihçesi & Felsefesi

Angular'ın neden bu hale geldiğini, hangi problemleri çözmek için tasarlandığını ve diğer framework'lerden nasıl ayrıştığını anlayacağız.



Hedef

Bu ders biraz "tarih dersi" gibi gelebilir, ama emin ol — kodun arkasındaki felsefeyi anlamadan, kod yazarken sürekli "neden böyle yapılmış?" sorusuyla mücadele edeceksin.

1

AngularJS — Her Şeyin Başlangıcı (2010)

Hikaye

2009 yılında Google'da çalışan Miško Hevery, ufak bir yan proje yapmaya başladı. Amacı: web tasarımcılarının HTML'i daha güçlü kullanmasını sağlamak. Ortaya çıkan şey: AngularJS.

Devrim Niteliğindeki Fikirler

```
<!-- AngularJS 2010 -->
<div ng-controller="UserCtrl">
  <input ng-model="name" />
  <p>Merhaba {{ name }}!</p>
</div>
```

Bu kod 2010'da BÜYÜK bir devrimdi. Çünkü:

- Two-way data binding — Input değişince name değişiyor, otomatik!
- Directives — HTML'i extend etme imkanı (ng-model, ng-repeat...)
- Dependency Injection — Backend kavramı frontend'e ilk kez geldi
- Two-way binding magic — DOM ve JS senkron, manuel update yok

AngularJS'in Sorunları

☐ Performance: Digest cycle her event sonrası tüm watcher'ları kontrol ediyordu. 1000 watcher'lı sayfada çok yavaştı.

☹ Karmaşık API: \$scope, \$rootScope, \$apply, \$digest, \$watch... Yeni başlayan için kabus.

☐ TypeScript Yok: JavaScript'te yazılıyordu. Büyük projelerde tip güvenliği yok.

❑ Modular Değil: Tüm uygulama tek bir bundle. Lazy loading karmaşık.

2 Büyük Yıkım ve Yeniden Doğuş — Angular 2 (2016)

❑ Cesur Karar

Google ekibi şu kararı verdi: "AngularJS'i fırlat. Sıfırdan yaz. Geriye uyumluluk yok." Bu çok tartışmalı bir karardı. Topluluk ikiye bölündü.

Felsefe Değişiklikleri

Konu	AngularJS	Angular 2
Dil	JavaScript	TypeScript
Mimari	MVC + Scope	Component-based
Module	angular.module(...)	NgModule
Change Detection	Digest cycle	Zone.js + zone-aware CD
Mobile	Zayıf	Mobile-first
DI	\$injector	Hierarchical Injector
Tooling	Yetersiz	Angular CLI

Neden TypeScript?

1. Ölçeklenebilirlik: 100K satırlık projede typo yazdığın anda IDE uyarır.
2. IntelliSense / Autocomplete: IDE tüm property'leri gösterir.
3. Refactoring: Property adı değişince TypeScript tüm kullanım yerlerini gösterir.
4. Dokümantasyon: `function getUserId(id: number): Promise<User | null>` — kod okuyarak ne yaptığını biliyorsun.

3 Angular'ın Felsefesi — "Opinionated" Olmak

Angular'ı diğer framework'lerden ayıran en büyük şey: opinionated olması.

"Opinionated" Ne Demek?

Opinionated framework: "Şunu şöyle yap, şu şekilde değil." Standartları dayatır.

Unopinionated framework: "Sen bilirsin, istediğin gibi yap."

Karşılaştırma

Soru	React	Angular
Routing nasıl?	React Router, Wouter... seç	Built-in Angular Router
State management?	Redux, Zustand... seç	NgRx Signal Store (resmi)
Forms?	react-hook-form, Formik... seç	Built-in Reactive Forms
HTTP?	fetch, axios... seç	Built-in HttpClient
Testing?	Jest, Vitest, Cypress... seç	Vitest (resmi)
CSS?	CSS modules, styled... seç	Component-scoped CSS
CLI?	Yok	Angular CLI (resmi)

Hangisi Daha İyi?

Bu felsefi bir soru. İkisinin de avantajları var:



React'in Esnekliği

Pro: Her ihtiyaca özel araç seç • Innovation hızlı • Hafif öğrenme eğrisi
Con: Her takım farklı stack • "Best practice" değişken • Junior'lar yolunu kaybedebilir



Angular'ın Standardı

Pro: Tüm projelerde tutarlı yapı • Senior her projeye hızlı adapte • "Doğru yol" belli • Enterprise için ideal
Con: Esneklik az • Innovation yavaş • Daha büyük initial bundle

Angular kurumsal odaklı. Büyük ekipler, uzun ömürlü projeler, yüksek code quality için tasarlandı.

4

Angular'ın Versiyon Tarihi — Önemli Dönüm Noktaları

Versiyon	Yıl	Önemli Yenilik
Angular 2	2016	Yeniden doğuş, TypeScript zorunlu
Angular 9	2020	Ivy renderer (büyük performance kazancı)
Angular 14-15	2022	Standalone components
Angular 16	2023	Signals (preview)
Angular 17	2023	@if/@for/@switch, @defer
Angular 19	2024	linkedSignal, resource API
Angular 20	2025	Zoneless stable
Angular 21	2025	Zoneless DEFAULT, Signal Forms, Vitest, MCP

Anahtar Noktalar

- Angular 9 — Ivy Renderer: Bundle size %40 küçüldü, build hızı 2x oldu.
- Angular 14-15 — Standalone Components: NgModule'ün ölüm fermanı. Boilerplate %50 azaldı.
- ✂ Angular 16 — Signals (Preview): Yeni reaktivite sistemi. Zone.js'in sonunun başlangıcı.
- Angular 17 — Modern Template Syntax: @if/@for, %90 daha okunabilir.
- Angular 21 — Zoneless Default: Sen şu anda bu versiyonu öğreniyorsun!

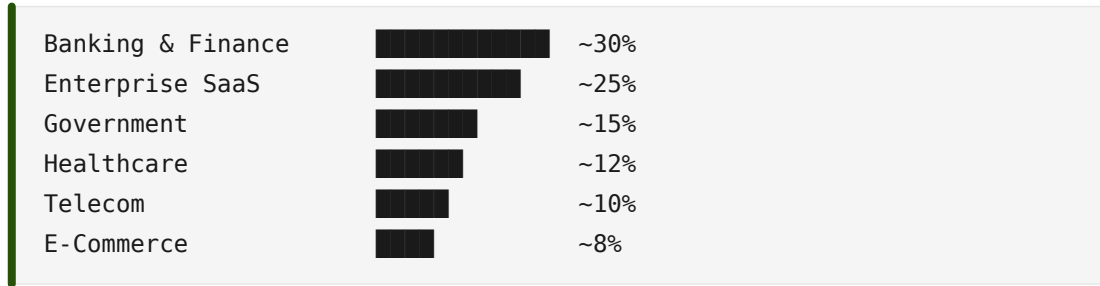
5 Angular Kim Tarafından Kullanılıyor?

Belki şöyle düşünebilirsin: "React her yerde, Angular niye öğreneyim?" Cevap: Angular kurumsal dünyanın kralı.

Angular Kullanan Büyük Şirketler

- 🇺🇸 Google — Gmail, AdWords, Maps console
- 🇺🇸 Microsoft — Office 365 web, Outlook web
- 🇺🇸 Forbes — Forbes.com
- 🇺🇸 Delta — Booking sistemi
- 🇺🇸 Upwork — Freelance platformu
- 🇩🇪 Deutsche Bank — Internal tools
- 🇫🇷 BNP Paribas — Banking platforms
- 🇬🇧 Wealthsimple — FinTech

Sektör Dağılımı



Çıkarım: Stabil, uzun vadeli, iyi maaşlı işler için Angular çok güçlü.

6 Angular'ın Geleceği

Kısa Vadeli (1-2 yıl)

- 🇺🇸 Signal Forms stable olacak (şu an experimental)
- 🇺🇸 Zoneless her yerde standart olacak
- 🇺🇸 MCP Server ile AI-assisted development güçlenecek
- 🇺🇸 Native Federation Module Federation'ın yerini alacak

Orta Vadeli (3-5 yıl)

- 🇺🇸 Server Components (React'tan ilham, Angular yorumu)
- 🇺🇸 Streaming SSR olgunlaşacak
- 🇺🇸 Edge Runtime desteği (Cloudflare Workers, Deno Deploy)
- 🇺🇸 Mobile-first improvements

7

Bu Eğitimde Neden Angular v21?

Angular v21, 5 yıl önceki Angular ile karşılaştırılamayacak kadar farklı. Modern, hızlı, type-safe, AI-friendly.

"Eski Kodu Öğrenmek Gerekmez Mi?"

Gerekirse öğrenirsin. Ama modern ile başlamak çok daha kolay. Çünkü:

- Eski Angular bilgisi modern Angular'a tam transfer olmaz
- Migration'ı zaten Modül 4'te göreceğiz
- Yeni kod yazarken modern best practice'leri direkt uygulayacaksın



Felsefe

"İyi bir Angular developer eski kodu okuyabilir, ama yeni kod yazarken modern olanı yazar."

□ Mini Quiz

- **AngularJS ve Angular 2+ arasındaki en büyük fark nedir?**

Cevap: AngularJS (1.x) ve Angular 2+ tamamen farklı framework'ler. Angular 2 sıfırdan yazıldı, TypeScript zorunlu hale geldi, MVC yerine component-based mimariye geçildi.

- **"Opinionated framework" ne demek?**

Cevap: Standartları dayatan, "doğru yolu" belirleyen framework. Angular routing, forms, HTTP, testing için resmi çözümler sunar — sen seçim yapmazsın.

- **Angular neden TypeScript zorunluluğu getirdi?**

Cevap: Büyük projelerde tip güvenliği, IDE desteği, refactoring kolaylığı ve self-documenting code için. Enterprise odaklı bir framework için TypeScript şart.

- **Angular 21'in en önemli yeniliği nedir?**

Cevap: Zoneless Change Detection'ın artık default olması. Bunun yanında Signal Forms (experimental), Vitest, Angular Aria, MCP Server.

- **Angular ne tür projeler için en uygundur?**

Cevap: Büyük takımlar, uzun ömürlü kurumsal projeler. Banking, healthcare, government, enterprise SaaS gibi sektörlerde özellikle güçlü.

- **NgModule'den Standalone'a geçiş neden önemliydi?**

Cevap: Boilerplate'in azaltılması, daha kolay tree-shaking, lazy loading'in basitleşmesi ve modern JavaScript module sistemine yaklaşma.

⚠ Anti-Patterns

- ✗ AngularJS ile Angular'ı karıştırmak — İki ayrı framework, bilgi transfer etmiyor
- ✗ "Angular = React'in alternatifi" sanmak — Hayır, ikisi farklı kategoriler
- ✗ Angular 1.x tutorial'larını takip etmek — Tamamen yanlış paradigma
- ✗ "Angular ölüyor" söylemine inanmak — Kurumsal dünyada hala büyüyor
- ✗ Modern Angular'ı eski Angular gibi yazmak — *ngIf, NgModule, class-based guard

□ Bu Dersten Çıkarman Gerekenler

- ✓ AngularJS ve Angular'ın iki ayrı framework olduğunu biliyorum
- ✓ Angular'ın opinionated olduğunu ve avantajlarını anlıyorum
- ✓ TypeScript'in Angular için neden zorunlu olduğunu kavradım
- ✓ Angular'ın enterprise odaklı felsefesini benimsiyorum
- ✓ Angular vs React vs Vue arasındaki bağlamsal farkları biliyorum
- ✓ v21'in neden özel bir versiyon olduğunu anladım

MODÜL 1 — DERS 1.3

Angular vs React vs Vue — Kod Karşılaştırması

"Angular, React, Vue arasındaki farklar nedir?" sorusunu kod yazmadan anlamak imkansız. Aynı uygulamayı üç framework'te yazıp karşılaştıracğız.



Hedef

Ders sonunda, ne zaman hangisini seçeceğini kendi gözünle göreceksin.

1

Karşılaştırma Senaryosu

Üç framework'te de aynı uygulamayı yazacağız: Mini Todo List.

Özellikler:

- ➡ Yeni todo ekleme
- ☐ Todo'yu tamamlandı işaretleme
- ☐ Todo silme
- ☐ İstatistik gösterimi
- ☐ Tamamlanmış todo'lar üstü çizili

Bu uygulama küçük ama her framework'ün temel API'lerini kullanıyor: state management, event handling, list rendering, conditional rendering, computed values.

2

React Versiyonu

```
import { useState } from 'react';

function TodoApp() {
  const [todos, setTodos] = useState([]);
  const [newTodoText, setNewTodoText] = useState('');

  const addTodo = () => {
    if (!newTodoText.trim()) return;
    setTodos([...todos, {
      id: Date.now(),
      text: newTodoText,
      completed: false
    }]);
    setNewTodoText('');
  };

  const toggleTodo = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    ));
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
  };

  const completedCount = todos.filter(t => t.completed).length;

  return (
    <div>
      <input
        value={newTodoText}
        onChange={(e) => setNewTodoText(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && addTodo()}
      />
      <button onClick={addTodo}>Ekle</button>

      <ul>
        {todos.map(todo => (
          <li key={todo.id} style={{
            textDecoration: todo.completed ? 'line-through' : 'none'
          }}>
            <input type="checkbox" checked={todo.completed}
              onChange={() => toggleTodo(todo.id)} />
            <span>{todo.text}</span>
            <button onClick={() => deleteTodo(todo.id)}>Sil</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

```
    )})  
  </ul>  
  
  {todos.length > 0 && (  
    <p>{completedCount} / {todos.length} tamamlandı</p>  
  )}  
</div>  
);  
}
```

React'in Felsefesi

"Sadece state'ini değiştir, ben re-render'ı hallederim. Ama dikkat: state immutable, side effect'ler useEffect içinde."

Dikkat Edilecek Noktalar

- useState hook — State yönetimi, her setState re-render tetikler
- JSX — JavaScript içinde HTML benzeri syntax, attribute'lar camelCase
- Immutable updates — Yeni array oluşturmak zorundasın, mutation yasak
- Inline expressions — JS doğrudan template'te
- {condition && <element/>} — JS truthy/falsy ile conditional
- key prop zorunlu — Her item için unique key

3 Vue Versiyonu

```
<script setup>
import { ref, computed } from 'vue';

const todos = ref([]);
const newTodoText = ref('');

const addTodo = () => {
  if (!newTodoText.value.trim()) return;
  todos.value.push({
    id: Date.now(),
    text: newTodoText.value,
    completed: false
  });
  newTodoText.value = '';
};

const toggleTodo = (id) => {
  const todo = todos.value.find(t => t.id === id);
  if (todo) todo.completed = !todo.completed;
};

const deleteTodo = (id) => {
  todos.value = todos.value.filter(t => t.id !== id);
};

const completedCount = computed(() =>
  todos.value.filter(t => t.completed).length
);
</script>

<template>
  <input v-model="newTodoText" @keydown.enter="addTodo" />
  <button @click="addTodo">Ekle</button>

  <ul>
    <li v-for="todo in todos" :key="todo.id"
      :class="{ completed: todo.completed }">
      <input type="checkbox" v-model="todo.completed" />
      <span>{{ todo.text }}</span>
      <button @click="deleteTodo(todo.id)">Sil</button>
    </li>
  </ul>

  <p v-if="todos.length > 0">
    {{ completedCount }} / {{ todos.length }} tamamlandı
  </p>
</template>
```

Vue'nun Felsefesi

"Geliştirici deneyimi her şeyden önemli. Template'in temiz olsun, mutation'la state değiştir, v- directive'leriyle güç al."

Dikkat Edilecek Noktalar

- `ref(...)` — Reactive state, `.value` ile okuma/yazma
- `computed(...)` — Türetilmiş değer
- `v-model` — Two-way binding
- `v-for`, `v-if` — Liste/conditional rendering
- `@click`, `@keydown.enter` — Event binding ve modifier'lar
- `<style scoped>` — Component-scoped CSS, otomatik
- Mutation OK — Vue'da mutation yapabilirsin (React'te yapamazsın)

4

Angular Versiyonu (v21 — Şu An Öğrendiğimiz)

```
import { Component, signal, computed } from '@angular/core';
import { FormsModule } from '@angular/forms';

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

@Component({
  selector: 'app-todo',
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="newTodoText"
      (keydown.enter)="addTodo()" />
    <button (click)="addTodo()">Ekle</button>

    <ul>
      @for (todo of todos(); track todo.id) {
        <li [class.completed]="todo.completed">
          <input type="checkbox" [checked]="todo.completed"
            (change)="toggleTodo(todo.id)" />
          <span>{{ todo.text }}</span>
          <button (click)="deleteTodo(todo.id)">Sil</button>
        </li>
      }
    </ul>

    @if (todos().length > 0) {
      <p>{{ completedCount() }} / {{ todos().length }} tamamlandı</p>
    }
  `,
})
export class TodoComponent {
  todos = signal<Todo[]>([]);
  newTodoText = '';

  completedCount = computed(() =>
    this.todos().filter(t => t.completed).length
  );

  addTodo() {
    if (!this.newTodoText.trim()) return;
    this.todos.update(list => [...list, {
      id: Date.now(),
      text: this.newTodoText,
      completed: false
    }]);
  }
}
```



```
    });  
    this.newTodoText = '';  
  }  
  
  toggleTodo(id: number) {  
    this.todos.update(list =>  
      list.map(todo => todo.id === id  
        ? { ...todo, completed: !todo.completed }  
        : todo)  
    );  
  }  
  
  deleteTodo(id: number) {  
    this.todos.update(list => list.filter(t => t.id !== id));  
  }  
}
```

Angular'ın Felsefesi

"Yapı, standardizasyon, type safety. Component nasıl yazılır net belli. 5 kişilik takım da, 500 kişilik takım da aynı şekilde yazar."

Dikkat Edilecek Noktalar

- signal() — Reactive state
- computed() — Türetilmiş değer
- TypeScript zorunlu — interface Todo ile şekil garanti
- Template syntax: [(ngModel)], (click), [class.x], [checked]
- Modern control flow: @for (track ZORUNLU), @if
- Component'in 3 parçası net — Decorator, class, template
- Decorator-based — @Component({...})

5 Yan Yana Karşılaştırma — Tablo

Konu	React	Vue	Angular
State definition	useState(0)	ref(0)	signal(0)
State okuma	count	count.value / count	count()
State yazma	setCount(5)	count.value = 5	count.set(5)
Computed	useMemo()	computed()	computed()
Two-way binding	value + onChange	v-model	[(ngModel)]
Event	onClick={fn}	@click="fn"	(click)="fn()"
Conditional	{cond && X}	v-if="cond"	@if (cond) { }
List	.map() + key	v-for + key	@for + track
Mutation izinli mi?	✗ Hayır	✓ Evet	✗ Hayır
CSS scoping	3rd party	scoped (built-in)	styles (built-in)
TypeScript	Opsiyonel	Opsiyonel	✓ Zorunlu

6 "Bu Kod Bana Ne Söylüyor?" — Felsefi Çıkarımlar

□ React: "JavaScript First"

React, JavaScript'in kendisine çok yakın. JSX bile aslında JavaScript. Conditional? && operatörü. List? .map(). State update? setState(...).

Avantaj: İyi JS biliyorsan, React'i 1 günde öğrenirsin.

Dezavantaj: Best practice'ler community-driven, "doğru yol" tartışmalı.

□ Vue: "Template First"

Vue, template syntax'a yatırım yapmış. v-model, v-if, v-for, :class... her şeyin direkt template'te bir karşılığı var.

Avantaj: HTML/CSS bilen biri için çok hızlı öğrenilir.

Dezavantaj: Vue'ya özel syntax çok (DSL).

❑ Angular: "Structure First"

Angular, yapı ve tutarlılık üstüne kurulu. Decorator, separate template, type-safe state, opinionated structure.

Avantaj: 100K satırlık projede hala temiz kalır.

Dezavantaj: Küçük proje için overkill, başlangıçta daha çok concept.

7

"Hangisini Öğrenmeliyim?" Kararı

Eğer Sen...	Tercih
❑ Kurumsal projelerde çalışacaksan	Angular
❑ Startup'ta hızlı iterasyon yapacaksan	React veya Vue
❑ Mobile app da yapacaksan	React (React Native)
❑ UI-heavy ürün geliştireceksen	React veya Vue
❑❑ İlk framework olarak öğreniyorsan	Vue (yumuşak eğri)
❑ İş bulma odaklıysan	Bölgeye bakar — Avrupa: Angular, ABD: React, Asya: Vue

Sen Şu An Angular'ı Öğreniyorsun. İyi mi Karar?

Evet, çok iyi karar. Çünkü:

- ✓ Senior Angular developer'lar çok aranıyor
- ✓ Kurumsal sektör stabil ve iyi maaşlı
- ✓ Angular kullananların çoğu uzun ömürlü projelerde
- ✓ TypeScript'in derinine ineceksin (her yerde işe yarar)
- ✓ Backend developer'larla çalışmak kolaylaşır
- ✓ Bir framework'ü gerçekten anlayan biri, diğerlerini hızlı öğrenir



Bilmen Gereken Sır

Bir framework'te uzmanlaştığında, diğerlerini öğrenmek 1-2 hafta. Konseptler aynı (state, component, reactivity), sadece syntax değişiyor.

□ Mini Quiz

- **React, Vue ve Angular arasındaki en temel felsefi fark nedir?**

Cevap: React = JavaScript first (esnek). Vue = Template first (HTML-centric). Angular = Structure first (opinionated, enterprise).

- **Angular'da neden mutation yapamıyoruz, Vue'da yapabiliyoruz?**

Cevap: Angular signals immutable update bekler. Vue'nun reactivity sistemi proxy-based, mutation'ı tracking edebiliyor. Angular daha "predictable", Vue daha "magic".

- **@for ... track, .map() + key, v-for + :key neden var?**

Cevap: List rendering yaparken framework'ün hangi item'ın hangisi olduğunu bilmesi gerekiyor (DOM diffing için). Unique identifier olmadan, item silindiğinde/eklendiğinde framework yanlış DOM update yapabilir.

- **Angular'ın bundle size'ı diğerlerinden daha büyük. Bu Angular'ı kötü yapıyor mu?**

Cevap: Hayır. Bundle size küçük projede önemli, büyük projede önemsizleşir. Angular zaten built-in çok şey getiriyor (router, forms, http, di). Ayrıca v21 zoneless ile bundle ~%50 küçüldü.

- **Senior Angular developer, React öğrenmek isterse ne kadar sürer?**

Cevap: Yaklaşık 1-2 hafta. Konseptler aynı, sadece syntax ve ekosistem farklı.

⚠ Anti-Patterns

- ✗ "Hangisi en iyi?" sorusunu sormak — Yanlış soru. Bağlama göre doğru framework değişir.
- ✗ Birden fazla framework'ü aynı projede kullanmak — Çok özel senaryolar dışında kabus.
- ✗ React'i Angular gibi yazmak veya tersi — Her birinin felsefesi farklı.
- ✗ Bundle size obsesyonu — Modern web'de 50KB fark çoğu zaman önemsiz.
- ✗ Performans karşılaştırma testlerine takılmak — Çoğu sentetik, gerçek projeleri yansıtmıyor.

□ Bu Dersten Çıkarman Gerekenler

- ✓ Üç framework'ün kod düzeyinde farklarını gördüm
- ✓ Her birinin felsefesini anlıyorum (JS-first, Template-first, Structure-first)
- ✓ Hangisinin hangi senaryoda uygun olduğunu biliyorum
- ✓ Angular'ın opinionated olmasının kod üzerinde etkilerini gördüm
- ✓ Bir framework'te uzmanlaşınca diğerlerinin kolay öğrenileceğini biliyorum
- ✓ Angular'ı öğrenmenin kariyer açısından doğru bir karar olduğunu hissediyorum

MODÜL 1 — DERS 1.4

Angular v21'in "Big Picture" Yenilikleri

v21'in getirdiği büyük yenilikleri kavramsal düzeyde tanıyacağız. Her birinin ne olduğu, neden önemli olduğu ve hangi problemi çözdüğü.



Önemli Not

Bu ders bir "tanıştırma" dersi. Kodda derinlemesine kullanmayı kendi modüllerinde göreceğiz. Şimdilik amaç: Angular v21 hakkında konuşurken hangi terimin neye karşılık geldiğini bilmek.

1

v21 Bir "Geçiş Versiyonu" Değil

Angular v21, 5 yıl önceki Angular ile karşılaştırılamayacak kadar farklı. Versiyonlar bazen küçük güncellemeler getirir. v21 öyle değil. Angular'ın mental modelini değiştiren birçok büyük yeniliği bir araya getiren versiyon.

Bu derste 6 büyük yeniliği inceleyeceğiz:

- ⚡ Zoneless Change Detection (artık default)
- 📁 Signal Forms (experimental)
- ♿ Angular Aria (accessibility primitives)
- 🧪 Vitest (Karma yerine)
- 🖥️ MCP Server (AI destekli geliştirme)
- 🎨 Tailwind CSS (yeni proje scaffold'ında)

2

Zoneless Change Detection — Devrimin Kalbi

Bu yenilik en önemlisi. Angular'ın çalışma mantığını temelinden değiştiriyor.

Önce: Zone.js Nedir?

Zone.js, 2014'ten beri Angular'ın gizli kalbi. JavaScript'in tüm async API'lerini monkey-patch eder, Angular'a haber verir.

```
setTimeout(() => {
  this.count++;
}, 1000);

// 1. Zone.js araya girer
// 2. "Async işlem başladı" Angular'a haber verir
// 3. Bittiğinde "şimdi her şeyi kontrol et" der
// 4. Angular tüm component'leri tekrar render eder
```

Bu sihir gibi çalışıyordu, ama 3 problemi vardı:



Sorun 1: Performance

Zone.js her async olayda tüm change detection cycle'ı tetikliyor. 100 component için 99 gereksiz kontrol.



Sorun 2: Bundle Size

Zone.js ~100KB. Her Angular uygulamasının yanında zorunlu olarak geliyor. Özellikle mobil için önemli.



Sorun 3: Predictability

Zone.js "magic" çalışıyor. Bir şey beklediğin gibi olmadığında debug etmek çok zor. Third-party library uyumsuzlukları.

Şimdi: Zoneless Dünya

Angular ekibi şu fikre çalıştı: "Ya değişiklikleri signals ile takip etsek? Tüm async olayları monkey-patch etmek yerine, gerçekten değişen yerleri bilelim?"

```
@Component({
  template: `<p>{{ count() }}</p>`
})
export class CounterComponent {
  count = signal(0);

  increment() {
    this.count.update(n => n + 1);
    // 1. Signal'in değerini değiştirir
    // 2. Signal "hey, ben değiştim" der
    // 3. Angular SADECE bu signal'i kullanan component'i günceller
    // 4. Diğer hiçbir component check edilmez
  }
}
```

Avantajları

- ✓ Daha hızlı — Sadece gerçekten değişen kısımlar render edilir
- ✓ Daha küçük bundle — Zone.js gereksiz, ~100KB tasarruf
- ✓ Daha öngörülebilir — "Magic" yok, signal değişince render olur
- ✓ Daha kolay debug — Component neden render oldu? net cevap
- ✓ Better SSR — Hydration daha hızlı

v21'de Ne Değişti?

```
// app.config.ts (v21 default)
export const appConfig: ApplicationConfig = {
  providers: [
    provideZonelessChangeDetection(), // ← v21 default
    provideRouter(routes),
    provideHttpClient(),
  ]
};
```

v21'de: `provideZonelessChangeDetection()` artık stable. Yeni proje oluştururken default olarak soruluyor.

3 Signal Forms — Forms'un Geleceği

Hikaye

Angular'da forms her zaman bir konu olmuş. Şu anda 2 yaklaşım var: Template-Driven Forms (eski, basit ama güçsüz) ve Reactive Forms (şu anki standart, güçlü ama verbose).

Reactive Forms (Şu Anki Standart)

```
form = this.fb.nonNullable.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', [Validators.required, Validators.minLength(8)]]
});
```

✓ Type-safe, ✓ Karmaşık formlar için ideal, ✗ Verbose, ✗ Signals ile uyumsuz

Gelecek: Signal Forms (v21'de Experimental)

"Form'lar zaten state. Signals state için. Bunları birleştirelim."

```
import { form, required, email, minLength } from '@angular/forms/signal
s';

@Component({...})
export class LoginComponent {
  // 1. State signal olarak
  data = signal({
    email: '',
    password: ''
  });

  // 2. Form, signal'den oluşturuluyor
  loginForm = form(this.data, (path) => {
    required(path.email);
    email(path.email);
    required(path.password);
    minLength(path.password, 8);
  });
}
```

Neden Devrimsel?

- ✓ Type-safe by default — TypeScript signal'in tipini biliyor
- ✓ Daha az boilerplate — FormControl, FormGroup yok
- ✓ Signal-native — Diğer signals ile sorunsuz çalışır

✓ Validation declarative — `required(path.email)` net

**Önemli Uyarı: v21'de Experimental**

Bu özellik henüz production-ready değil. API değişebilir. Strateji: Yeni projede Reactive Forms kullan, Signal Forms'u göz at. 1-2 yıl içinde stable olacak.

4

Angular Aria — Accessibility Devrimi

Hikaye

Accessibility (a11y) bugüne kadar Angular dünyasında 3rd party library'lerle çözülmüyordu. Material kullanmadan accessible bir combobox veya dropdown yazmak çok zordu.

Çözüm: Angular Aria

v21 ile Angular ekibi @angular/aria paketini çıkardı. Headless accessibility primitives sağlıyor: "Sana accessible component'ların mantığını vereyim. Görünüşü sen tasarla."

```
import { CdkAccordion } from '@angular/aria';

@Component({
  imports: [CdkAccordion],
  template: `
    <div cdkAccordion>
      <div cdkAccordionItem>
        <button cdkAccordionTrigger>Section 1</button>
        <div cdkAccordionPanel>Content...</div>
      </div>
    </div>
  `,
  styles: [`
    /* Sen styling'i hallediyorsun */
    [cdkAccordionPanel] { padding: 1rem; }
  `]
})
export class AccordionComponent {}
```

Headless Component Nedir?

Headless = Sadece davranış, görünüş yok.

Yaklaşım	Davranış	Görünüş	Esneklik
Angular Material	Built-in	Built-in (Material)	Düşük
Headless (Angular Aria)	Built-in	Sen yazıyorsun	Yüksek
Sıfırdan yazma	Sen yazıyorsun	Sen yazıyorsun	Çok yüksek (riskli)

Sağlanan Primitives

- ☐ Accordion — Açılır/kapanır panel'ler
- ☐ Combobox — Autocomplete input
- ☐ Listbox — Selectable list
- ☐ Menu — Dropdown menu
- ☐ Tabs — Tab navigation
- ☐ Tree — Hierarchical tree

Hepsi: Klavye navigasyonu, ARIA attributes otomatik, Screen reader uyumlu, Focus management.

5 Vitest — Karma'nın Sonu

Hikaye

Angular her zaman Karma + Jasmine ile test edilirdi. Sorunları: yavaş (browser başlatma), eski (2014'ten beri update yok), karmaşık config, kötü watch mode.

v21 Çözümü: Vitest Default

Yeni Angular projeleri artık Vitest ile geliyor. Karma artık opt-in.

```
import { TestBed } from '@angular/core/testing';
import { describe, it, expect } from 'vitest';

describe('CounterComponent', () => {
  it('should increment count', () => {
    const fixture = TestBed.createComponent(CounterComponent);
    fixture.componentRef.setInput('initial', 0);
    fixture.detectChanges();

    fixture.componentInstance.increment();
    expect(fixture.componentInstance.count()).toBe(1);
  });
});
```

Avantajları

- ✓ 5-10x daha hızlı — Browser yerine jsdom
- ✓ Hot reload — Test değişince anında çalışır
- ✓ Modern API — Jest'e benzer (büyük community)
- ✓ Better mocking — vi.fn(), vi.mock(), vi.useFakeTimers()
- ✓ Built-in coverage — Ekstra plugin gerek yok

Migration

```
ng generate @angular/core:vitest-migration
```

Otomatik olarak karma.conf.js siler, vitest.config.ts ekler, test dosyalarını günceller.

6 MCP Server — AI Çağına Entegrasyon

Hikaye

2024-2025 AI'nın yazılım geliştirmeye girdiği yıllar. Anthropic Model Context Protocol (MCP) standartını çıkardı. AI agent'lar ile araçlar arasında iletişim için.

Angular ekibi: "Angular CLI'yi MCP Server yapalım. AI'lar Angular'a doğrudan bağlanabilsin, doğru Angular kodu yazabilsin."

Kullanımı

```
# Angular MCP Server'ı başlat
ng mcp

# Cursor'a entegre et:
# .cursor/mcp.json
{
  "mcpServers": {
    "angular": {
      "command": "npx",
      "args": ["-y", "@angular/cli", "mcp"]
    }
  }
}
```

Neden Devrimsel?



Eski Dünya (MCP'siz)

Cursor'a yazıyorsun: "Angular'da reactive form yap." Cursor 2 yıl eski training data'dan kod üretiyor — NgModule, *ngIf kullanan eski kod.



Yeni Dünya (MCP ile)

Cursor MCP üzerinden Angular CLI'ye bağlanıyor. find_examples ile kürate edilmiş örnekleri alıyor. v21 standartlarında kod üretiyor (signals, standalone, @if/@for).

Kullanılabilir MCP Tools

- find_examples — Angular ekibinin kürate ettiği örnekler
- get_best_practices — En güncel best practice'ler
- search_documentation — Resmi dokümandan arama
- ai_tutor — Etkileşimli AI öğretmen

- ♻ modernize — Eski Angular kodunu v21'e dönüştürme



Sen Şu Anda Claude ile Çalışıyorsun

Angular MCP Server'ı kurarsan: Claude Code (terminal), Cursor (IDE), VS Code Copilot — hepsi doğrudan Angular'a bağlanır ve modern, doğru kod üretir.

7 Tailwind CSS — Yeni Default Styling

Tailwind Felsefesi

"Önceden tanımlanmış utility class'lar ile doğrudan HTML'de styling yap. Custom CSS dosyasına gitme."

```
<!-- Eski yaklaşım -->
<div class="user-card">
  <img class="user-avatar" />
  <h3 class="user-name">John</h3>
</div>
<style>
  .user-card { ... }
  .user-avatar { ... }
  .user-name { ... }
</style>

<!-- Tailwind yaklaşımı -->
<div class="p-4 bg-white rounded-lg shadow flex items-center gap-3">
  <img class="w-12 h-12 rounded-full" />
  <h3 class="text-lg font-semibold">John</h3>
</div>
```

Neden Default'a Eklendi?

Önceki versiyonlarda Tailwind kurmak karmaşıktı. v21'de: `ng new my-app` → "Use Tailwind CSS?" → Yes → Hepsi otomatik kurulur.

Avantaj-Dezavantaj

- ✓ Hızlı prototyping
- ✓ Tutarlı design system (spacing, colors auto)
- ✓ Küçük production CSS (purging)
- ✓ Responsive design kolaylığı
- ✗ HTML kalabalıklaşır (uzun class listeleri)
- ✗ Öğrenme eğrisi (class isimlerini ezberlemek)

☐ **Senin Stratejin**

Modül 1-12: Vanilla SCSS (temellere odaklanmak için)

Modül 13+: Angular Material (UI library)

Capstone: Tailwind opsiyonu açık. Şu an Tailwind öğrenmene gerek yok — öncelik Angular.

8

Diğer Önemli ama Daha Küçük Yenilikler

httpResource() Stable

Service yazmadan HTTP isteği:

```
users = httpResource<User[]>(() => ({  
  url: '/api/users',  
  params: { q: this.search() }  
}));
```

Component-Level Hydration Control

```
@Component({  
  hydrate: 'never' // veya 'always', 'whenIdle', 'whenVisible'  
})
```

@switch Fall-Through (v21.1)

```
@switch (status) {  
  @case ('pending')  
  @case ('processing') { <app-loading /> }  
  @case ('completed') { <app-success /> }  
}
```

Improved Error Messages

Compiler hataları artık çok daha açıklayıcı.

9

Bu Yenilikler Senin İçin Ne Demek?

Pratik Anlamda

- ✓ Daha hızlı uygulamalar — Zoneless ile bundle ~%50 küçük
- ✓ Daha temiz kod — Signal Forms ile boilerplate az
- ✓ Daha kolay test — Vitest ile %5-10x hızlı
- ✓ Daha iyi a11y — Angular Aria ile sıfırdan accessible
- ✓ AI ile çalışma — MCP Server ile modern kod
- ✓ Hızlı styling — Tailwind opsiyonel

Mental Model Değişimi



Eski Angular Düşüncesi

Async olay → Zone.js → Tüm app güncellenir
NgModule → declare et → export et → import et
*ngIf → ng-template → Else case



Yeni Angular Düşüncesi

Signal değişir → Sadece o yerler güncellenir
Standalone component → Import et → Kullan
@if → @else → @else if (HTML benzeri)

Migration Endişesi

Eski projeler ne olacak? Geriye uyumluluk var. Migration tool'larla:

```
ng update @angular/cli @angular/core          # Versiyona güncelle
ng generate @angular/core:zoneless-migration   # Zoneless'a geç
ng generate @angular/core:control-flow         # *ngIf → @if
ng generate @angular/core:standalone           # NgModule → Standalone
ng generate @angular/core:signal-input-migration # @Input → input()
ng generate @angular/core:vitest-migration     # Karma → Vitest
```

Bu komutlar otomatik refactor ediyor. Manuel iş minimum. Bu yüzden modern Angular öğrenmek eski Angular'ı öğrenmekten daha akıllıca.

□ Mini Quiz

- **Zone.js'in 3 ana sorunu neydi?**

Cevap: 1) Performance — gereksiz change detection. 2) Bundle size — ~100KB ek yük. 3) Predictability — "magic" davranış, debug zor.

- **Zoneless Change Detection'da render ne zaman tetiklenir?**

Cevap: Bir signal değiştiğinde. Angular sadece o signal'e bağlı component'leri günceller.

- **Signal Forms ile Reactive Forms arasındaki temel fark nedir?**

Cevap: Reactive Forms ayrı bir state sistemi. Signal Forms zaten signal'lerden oluşan state'i form olarak kullanır — daha az boilerplate, signals ile native uyum.

- **Angular Aria neden "headless" diye anılıyor?**

Cevap: Sadece accessibility davranışını sağlar (klavye, ARIA, focus). Görünüşü (CSS) sen yazarsın. Her tasarıma uyar.

- **Vitest, Karma'ya göre hangi avantajları sunar?**

Cevap: 5-10x daha hızlı, modern API, built-in coverage, daha iyi watch mode, daha kolay mocking.

- **MCP Server'ın temel amacı nedir?**

Cevap: AI agent'ların Angular CLI'ye bağlanıp güncel ve doğru Angular kodu üretmesini sağlamak. Training data eskimişlik sorununu çözer.

⚠ Anti-Patterns

- ✗ Zoneless'ı zone-aware library'lerle karıştırmak — Eski library'ler sorun çıkarabilir
- ✗ Production'da Signal Forms kullanmak — Henüz experimental, Reactive Forms tercih et
- ✗ MCP Server'sız AI ile Angular kodu üretmek — Eski kod alma riski yüksek
- ✗ Karma'da takılı kalmak — Vitest migration'ı yap, %5-10x hız kazanırsın
- ✗ Sıfırdan accessible component yazmaya çalışmak — Angular Aria primitives kullan

□ Bu Dersten Çıkarman Gerekenler

- ✓ Zoneless Change Detection'ın ne olduğunu ve neden devrim olduğunu biliyorum
- ✓ Signal Forms'un Reactive Forms'tan farkını anlıyorum
- ✓ Angular Aria'nın headless component yaklaşımını kavradım
- ✓ Vitest'in Karma'ya göre avantajlarını biliyorum
- ✓ MCP Server'ın AI çağındaki rolünü anladım

- ✓ Tailwind CSS'in v21'de opsiyonel olarak geldiğini biliyorum
- ✓ v21 mental model'ini hissetmeye başladım — signals merkez

MODÜL 1 — DERS 1.5

Geliştirme Ortamı Kurulumu

Bu derste kodlamaya hazır bir geliştirme ortamı kuracağız. Node.js, Angular CLI, VS Code, gerekli uzantılar ve ilk Angular projemizi başlatacağız.



Pratik Ders

Bu pratik bir ders. Anlatacağım, sen yapacaksın. Her adımı bilgisayarında uygulamayı bekliyorum.

1

Geliştirme Ortamımızda Neler Olacak?

Bir Angular geliştiricisinin olmazsa olmaz araçları:

- ☐ Node.js (LTS) — JavaScript runtime, npm dahil
- ☐ npm — Package manager (Node ile gelir)
- ☐ Angular CLI — Angular komut satırı aracı
- ☐ VS Code — Code editor
- ☐ Angular Language Service — IDE desteği
- ☐ ESLint + Prettier — Code quality + formatting
- ☐ Git — Version control
- ☐ Angular DevTools — Browser extension
- ☐ Modern bir tarayıcı — Chrome / Edge / Firefox

2

Node.js Kurulumu

Neden Node.js?

Angular bir frontend framework olsa da, build araçları Node.js üzerinde çalışır. Angular CLI, npm paketleri, dev server — hepsi Node.js gerektirir.

Hangi Sürüm?

Angular v21 için Node.js 20.x veya 22.x (LTS) öneriliyor.

i LTS Nedir?

LTS = Long Term Support. Stabil, güvenilir, uzun vadeli destek alan sürüm. Mutlaka LTS yükle, "Current" sürümü değil.

Kontrol: Node.js Yüklü mü?

```
node --version
```

Çıktı yorumlama:

- ✓ v20.18.0 ✓ Yüklü ve uygun
- ⚠ v18.19.0 ⚠ Çalışır ama eski
- ✗ v16.20.0 ✗ Çok eski
- ✗ command not found ✗ Yüklü değil

Windows Kurulum

Yöntem 1: Resmi Installer (Kolay)

- <https://nodejs.org> adresine git
- LTS versiyonu indir (yeşil button)
- .msi dosyasını çalıştır, varsayılanları kabul et

Yöntem 2: nvm-windows (Önerilen)

```
# https://github.com/coreybutler/nvm-windows/releases adresinden installer indir
nvm install lts
nvm use lts
node --version
```

macOS Kurulum

```
# Homebrew ile (önerilen)
brew install node@20

# Veya nvm ile
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.zshrc
nvm install --lts
nvm use --lts
```

Linux Kurulum (Ubuntu/Debian)

```
# NodeSource repo (önerilen)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Kontrol
node --version
npm --version
```

Doğrulama

```
node --version    # v20.x.x veya v22.x.x görmeli
npm --version     # 10.x.x görmeli
```

3 Angular CLI Kurulumu

Angular CLI Nedir?

Komut satırı aracı. Yeni proje oluşturma, component generate etme, build alma, test çalıştırma — hepsi Angular CLI ile.

Kurulum

```
npm install -g @angular/cli@latest
```

-g flag'i global demek — terminal'de her yerden ng komutu kullanabileceksin.

Doğrulama

```
ng version
```

Şöyle bir çıktı görmelisin:

```
Angular CLI: 21.0.0  
Node: 20.18.0  
Package Manager: npm 10.x.x  
OS: ...
```



Hata: "ng: command not found"

Bu olursa terminal'i kapat ve yeniden aç. Hala olmuyorsa PATH'e ekleme yapman gerekiyor:

macOS/Linux: `export PATH="$PATH:$(npm config get prefix)/bin"` satırını `.zshrc/.bashrc`'ye ekle.

4 VS Code Kurulumu ve Uzantılar

Neden VS Code?

Angular için en yaygın IDE. Microsoft'un ücretsiz editörü. Angular Language Service ve diğer uzantılarla mükemmel uyum.

Kurulum

<https://code.visualstudio.com> adresinden işletim sistemine uygun versiyonu indir ve kur.

Olmazsa Olmaz Uzantılar

VS Code aç → `Ctrl+Shift+X` → Aşağıdakileri arayıp yükle:

1□ Angular Language Service

Template'lerde autocomplete, hata kontrolü, "Go to definition"

2□ ESLint

Kod kalitesi kontrolü, real-time linting

3□ Prettier - Code Formatter

Otomatik kod formatlama

4□ EditorConfig for VS Code

Takım çalışmalarında tutarlı formatlama

Faydalı Uzantılar

- Material Icon Theme — Dosya/klasör ikonları
- Error Lens — Hataları satır içinde gösterir
- Path Intellisense — Import path autocomplete
- Auto Rename Tag — HTML tag otomatik yeniden adlandırma
- GitLens — Git geçmişi, blame, satır bazında commit
- Angular Snippets — Hızlı Angular template'leri

Toplu Kurulum

```
code --install-extension Angular.ng-template
code --install-extension dbaeumer.vscode-eslint
code --install-extension esbenp.prettier-vscode
code --install-extension EditorConfig.EditorConfig
code --install-extension PKief.material-icon-theme
code --install-extension usernamehw.errorlens
code --install-extension christian-kohler.path-intellisense
code --install-extension formulahendry.auto-rename-tag
code --install-extension eamodio.gitlens
code --install-extension johnpapa.Angular2
```


5 VS Code Ayarları

VS Code'un default ayarları iyi, ama Angular için birkaç custom ayar öneririm. Ctrl+Shift+P → "Preferences: Open User Settings (JSON)" → Aşağıdakini ekle:

```
{
  // Editor
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "editor.bracketPairColorization.enabled": true,

  // Files
  "files.eol": "\n",
  "files.insertFinalNewline": true,
  "files.trimTrailingWhitespace": true,
  "files.autoSave": "onFocusChange",

  // ESLint
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },

  // Prettier
  "prettier.singleQuote": true,
  "prettier.printWidth": 100,
  "prettier.semi": true,

  // Material Icons
  "workbench.iconTheme": "material-icon-theme",

  // TypeScript
  "typescript.preferences.importModuleSpecifier": "relative",
  "typescript.updateImportsOnFileMove.enabled": "always"
}
```

Ne yaptık?

- ✓ Format on save — Save'e bastığında kod otomatik formatlanır
- ✓ ESLint auto-fix — Save'de düzeltilebilir hatalar otomatik düzeltilir
- ✓ Single quote — 'abc' standardı
- ✓ Tab size 2 — Angular topluluğu standardı
- ✓ Auto save on focus change — Pencere değiştirdiğinde otomatik kaydet

6

Git Kurulumu

Version control olmadan profesyonel geliştirme olmaz.

Git Yüklü mü?

```
git --version
```

Yüklü Değilse

```
# Windows: https://git-scm.com adresinden indir
# macOS:
brew install git

# Linux:
sudo apt-get install git
```

İlk Konfigürasyon

```
git config --global user.name "Adın Soyadın"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main
```



Email Önemli

Email, GitHub hesabınla aynı olsun (commit'lerin GitHub'da sana atanır).

7

Angular DevTools Browser Extension

Bu olmazsa olmaz. Component tree görmek, signal graph incelemek, profiling yapmak için.

Chrome / Edge için

- Chrome Web Store: <https://chromewebstore.google.com>
- "Angular DevTools" ara
- "Add to Chrome" tıkla

Test

- Bir Angular sitesi aç (örn: <https://angular.dev>)
- F12 ile DevTools aç
- Üst sekme barında "Angular" sekmesi varsa kurulum başarılı ✓

8

İlk Angular Projemizi Oluşturma

Şimdi tüm aletlerimiz hazır. Hadi gerçek bir proje yaratıp çalıştıralım!

Adım 1: Proje Klasörü Hazırla

```
# macOS / Linux
cd ~/Documents
mkdir angular-egitim
cd angular-egitim

# Windows
cd C:\Users\YourName\Documents
mkdir angular-egitim
cd angular-egitim
```

Adım 2: Yeni Angular Projesi Oluştur

```
ng new hello-angular
```

Angular CLI sana sorular soracak:



Stylesheet format?

SCSS seç. Vanilla CSS'in tüm özellikleri + nesting, variables, mixins.



Server-Side Rendering (SSR)?

No seç. SSR'ı Modül 25'te göreceğiz.



Zoneless Change Detection?

Yes seç! v21'in default'u ve gelecekteki standart bu.



Tailwind CSS?

No seç. Vanilla SCSS ile başlayacağız, Material'a Modül 13'te geçeceğiz.

Adım 3-5: Projeyi Aç ve Başlat

```
cd hello-angular
code .          # VS Code'da aç
ng serve        # Dev server başlat
```

Birkaç saniye bekle. Şöyle bir çıktı göreceksin:

```
Application bundle generation complete. [1.234 seconds]

Watch mode enabled. Watching for file changes...
→ Local:    http://localhost:4200/
```

Adım 6: Tarayıcıda Aç

Tarayıcıda <http://localhost:4200> adresine git.



Tebrikler!

"hello-angular app is running!" yazısını görmelisin. İlk Angular uygulamanız çalışıyor!

9

Hızlı Bir Düzenleme — Hot Reload Görelim

Şimdi gerçek bir geliştirici gibi davranalım.

Adım 1: src/app/app.html dosyasını aç

TÜMÜNÜ silin. Yerine şunu yapıştırın:

```
<main style="text-align: center; padding: 4rem;">
  <h1 style="font-size: 3rem; color: #dd0031;">
    Merhaba Angular v21!
  </h1>
  <p>Bu benim ilk Angular uygulamam.</p>
  <button (click)="count.set(count() + 1)">
    Tıkla beni: {{ count() }}
  </button>
</main>
```

Adım 2: src/app/app.ts dosyasını düzenle

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.scss']
})
export class App {
  protected readonly title = signal('hello-angular');
  protected readonly count = signal(0); // ← Ekle bunu
}
```

Adım 3: Save (Ctrl+S / Cmd+S)

Tarayıcıya bak — otomatik refresh olduğunu göreceksin! Butona tıkla, sayı artıyor.



Az önce kullandığın şey:

- Bir signal (count = signal(0))
- Event binding ((click))
- Interpolation ({{ count() }})
- Hot Module Replacement (HMR) — kod değişince otomatik refresh

Modül 4 ve 5'te bunları derinlemesine işleyeceğiz.

10 Angular DevTools'u İlk Kez Kullanma

Tarayıcı hala açık. Şimdi DevTools'u açalım.

- F12 ile DevTools aç
- Üst sekme barında "Angular" sekmesine geç
- Solda component tree görürsün: App → RouterOutlet
- App'e tıkla. Sağda state görünür: title, count
- Butona tıkla. DevTools'taki count değeri otomatik güncellenir!

Profiler'ı Dene

- Üstte "Profiler" sekmesi → "Start recording"
- Birkaç kez butona tıkla
- "Stop recording"
- Karşına flame graph gelir — render time analizi



Profil Etmek

Bu büyük projelerde performance bottleneck'leri bulmak için altın değerinde bir araç.

11 Geliştirme Workflow'u

Tipik bir Angular geliştirme akışın şöyle olacak:

1. Terminal aç
2. cd proje-klasörü
3. ng serve → Dev server başlat
4. VS Code'da kod yaz → Save → Otomatik refresh
5. Browser'da test et
6. Angular DevTools ile debug et
7. git commit, git push → Versiyon kontrolü

12 Sık Karşılaşılan Problemler

Problem 1: "Port 4200 is already in use"

Sorun: Başka Angular projesi çalışıyor.

Çözüm: ng serve --port 4201

Problem 2: "Cannot find module '@angular/...'"

Sorun: Bağımlılıklar tam yüklenmedi.

Çözüm: `rm -rf node_modules package-lock.json && npm install`

Problem 3: VS Code'da kırmızı çizgiler ama derleme başarılı

Sorun: TypeScript Server takılmış.

Çözüm: `Ctrl+Shift+P` → "TypeScript: Restart TS Server"

Problem 4: "ng: command not found" (kurulumdan sonra)

Sorun: PATH'e eklenmemiş.

Çözüm: Yeni terminal aç. Hala olmuyorsa Bölüm 4'teki PATH ayarını yap.

13 Bonus — Pratik Komutlar Sözlüğü

Eğitim boyunca sık kullanacağın komutlar:

Proje yönetimi

```
ng new <proje-adı>          # Yeni proje
ng serve                    # Dev server (default 4200)
ng serve --port 4201        # Custom port
ng serve --open              # Otomatik tarayıcı aç
ng build                    # Development build
ng build --configuration production
ng test                     # Test runner
```

Code generation

```
ng generate component <name> # ng g c <name>
ng generate service <name>   # ng g s <name>
ng generate directive <name> # ng g d <name>
ng generate pipe <name>      # ng g p <name>
ng generate guard <name>     # ng g g <name>
ng generate interceptor <name>
```

Bağımlılık yönetimi

```
npm install          # Tüm paketleri yükle
npm install <paket>  # Yeni paket ekle
npm install -D <paket> # Dev dependency
npm uninstall <paket> # Paket sil
npm update           # Güncelle
```

Angular güncelleme

```
ng update          # Hangileri güncellenebilir
ng update @angular/cli @angular/core # Angular'ı güncelle
ng mcp             # MCP Server (v21)
```


📝 Mini Quiz

- **Angular v21 için minimum Node.js sürümü nedir?**

Cevap: Node.js 20.x veya 22.x (LTS). Eski sürümler çalışmayabilir.

- **Angular CLI nasıl global olarak yüklenir?**

Cevap: `npm install -g @angular/cli@latest`

- **VS Code'da Angular için olmazsa olmaz uzantı hangisidir?**

Cevap: Angular Language Service (Angular ekibinin resmi uzantısı). Template autocomplete ve hata kontrolü için şart.

- **ng serve komutu ne yapar ve default port nedir?**

Cevap: Development server'ı başlatır, hot reload aktif eder. Default port: 4200.

- **Angular DevTools nedir, neden gerekli?**

Cevap: Browser extension. Component tree görmek, state inspect, change detection profiling için. Debug etmek olmazsa olmaz.

- **ng new sırasında "Zoneless Change Detection?" sorusuna ne diyeceksin?**

Cevap: Yes! v21'in default'u ve gelecekteki standart bu.

- **Hot Module Replacement (HMR) ne demek?**

Cevap: Kod değişikliklerinin tarayıcıda otomatik yansması. Save'e bastığında manuel refresh gerekmez.

⚠️ Anti-Patterns

- ✗ Global Angular CLI versiyonu ile proje versiyonu uyumsuzluğu — Hep güncel tut.
- ✗ node_modules ve dist klasörlerini git'e commit etmek — .gitignore'da zaten var.
- ✗ VS Code uzantısı kurmadan kod yazmaya çalışmak — Angular Language Service şart.
- ✗ Format on save kapalı — Kod tutarsızlığı, takım çalışmasında sorun.
- ✗ Angular DevTools'u görmezden gelmek — Console.log primitive bir yaklaşım.
- ✗ Eski Node sürümü kullanmak (v16 ve altı) — Güvenlik açıkları.

📝 Bu Dersten Çıkarman Gerekenler

- ✓ Node.js LTS kuruldu, sürümü doğruladım
- ✓ Angular CLI v21 global olarak yüklü
- ✓ VS Code ve önemli uzantılar hazır
- ✓ Git konfigüre edildi
- ✓ Angular DevTools browser extension'ı yüklü

- ✓ İlk Angular projem çalışıyor (hello-angular)
- ✓ Hot reload'un nasıl çalıştığını gördüm
- ✓ Temel Angular CLI komutlarını biliyorum

☐ Modül 1 Tamamlandı!

Tebrikler! Modül 1'i başarıyla tamamladın. Bu modülde Angular dünyasının kapısını araladık.

Neler Öğrendin?

Ders 1.1

Frontend framework'lerinin neden var olduğunu, vanilla JS'in sınırlarını, SPA konseptini, component-based mimariyi ve reactivity'yi öğrendin.

Ders 1.2

Angular'ın AngularJS'ten v21'e olan yolculuğunu, opinionated felsefesini, neden TypeScript zorunlu olduğunu öğrendin.

Ders 1.3

Angular vs React vs Vue'yu kod düzeyinde karşılaştırdın, her birinin felsefesini gördün.

Ders 1.4

v21'in büyük yeniliklerini (Zoneless, Signal Forms, Vitest, MCP, Angular Aria, Tailwind) kavramsal olarak öğrendin.

Ders 1.5

Geliştirme ortamını kurdun, ilk Angular uygulamayı çalıştırdın, hot reload'u gördün, DevTools'u kullandın.



Sıradaki Modül: TypeScript Refresher

Modül 2 — TypeScript'in Angular'da sıkça kullanılan özelliklerini hızlıca gözden geçireceğiz: Tipler, interfaces, generics, decorators, utility types, type guards. Bu konuları zaten biliyor olabilirsin, ama Angular bağlamında nasıl kullanılacağına odaklanacağız.